Data Quality Engineering using Databricks DQX

1. Objective

The objective of this assignment is to implement a complete data quality pipeline using **Databricks DQX** on a tampered aviation dataset. The dataset contains intentionally introduced issues including:

- Null values and invalid data types
- Out-of-range numeric values
- Future dates
- Duplicate records

The workflow includes:

- Profiling raw data
- Defining quality expectations
- Detecting bad records
- Routing failed data into quarantine
- Remediation and validation

2. Dataset Overview

The dataset contains aviation landing statistics with the following attributes:

- Activity Period
- Activity Period Start Date
- Operating Airline

- Airline IATA Codes
- GEO Summary and Region
- Aircraft Type and Model
- Landing Count
- Total Landed Weight

After loading the raw dataset into Databricks, a trimmed dataframe **dfLandStat_trimmed** was created to standardize column names, remove unnecessary fields, and cast numeric and date values.

```
05:40 PM (<1s)                                    8
dfLandStat_trimmed.printSchema()

root
 |-- Activity_Period: string (nullable = true)
 |-- Activity_Period_Start_Date: date (nullable = true)
 |-- Operating_Airline: string (nullable = true)
 |-- Operating_Airline_IATA_Code: string (nullable = true)
 |-- Published_Airline: string (nullable = true)
 |-- Published_Airline_IATA_Code: string (nullable = true)
 |-- GEO_Summary: string (nullable = true)
 |-- GEO_Region: string (nullable = true)
 |-- Landing_Aircraft_Type: string (nullable = true)
 |-- Aircraft_Body_Type: string (nullable = true)
 |-- Aircraft_Manufacturer: string (nullable = true)
 |-- Aircraft_Model: string (nullable = true)
 |-- Aircraft_Version: string (nullable = true)
 |-- Landing_Count: integer (nullable = true)
 |-- Total_Landed_Weight: double (nullable = true)
```

05:40 PM (3s) 7 Python

```python
#Trim the dataframe
dfLandStat_trimmed = dfLandStat.drop("data_as_of","data_loaded_at")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Activity Period","Activity_Period")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Activity Period Start Date",
"Activity_Period_Start_Date")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Operating Airline",
"Operating_Airline")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Operating Airline IATA Code",
"Operating_Airline_IATA_Code")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Published Airline",
"Published_Airline")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Published Airline IATA Code",
"Published_Airline_IATA_Code")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("GEO Summary","GEO_Summary")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("GEO Region","GEO_Region")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Landing Aircraft Type",
"Landing_Aircraft_Type")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Aircraft Body Type",
"Aircraft_Body_Type")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Aircraft Manufacturer",
"Aircraft_Manufacturer")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Aircraft Model","Aircraft_Model")
dfLandStat_trimmed = dfLandStat_trimmed.withColumnRenamed("Aircraft Version",
"Aircraft_Version")
# Clean and cast numeric columns (remove commas first)
from pyspark.sql.functions import col, trim, regexp_replace, to_date
dfLandStat_trimmed = dfLandStat_trimmed.withColumn(
    "Landing_Count",
    regexp_replace(trim(col("Landing Count")), ",", "").cast("int")
)

dfLandStat_trimmed = dfLandStat_trimmed.withColumn(
    "Total_Landed_Weight"
```

05:40 PM (3s) 7 Python

```python
# Clean and cast numeric columns (remove commas first)
from pyspark.sql.functions import col, trim, regexp_replace, to_date
dfLandStat_trimmed = dfLandStat_trimmed.withColumn(
    "Landing_Count",
    regexp_replace(trim(col("Landing Count")), ",", "").cast("int")
)

dfLandStat_trimmed = dfLandStat_trimmed.withColumn(
    "Total_Landed_Weight",
    regexp_replace(trim(col("Total Landed Weight")), ",", "").cast("double")
)
# Parse date column using correct format
dfLandStat_trimmed = dfLandStat_trimmed.withColumn(
    "Activity_Period_Start_Date",
    to_date(col("Activity_Period_Start_Date"), "M/d/yyyy")
)

# Drop original raw numeric columns
dfLandStat_trimmed = dfLandStat_trimmed.drop("Landing Count","Total Landed Weight")

display(dfLandStat_trimmed)
```

See performance (1)

dfLandStat_trimmed: pyspark.sql.connect.dataframe.DataFrame = [Activity_Period: string, Activity_Period_Start_Date: date ... 13 more fields]

Table

	Activity_Period	Activity_Period_Start_Date	Operating_Airline	Operating_Airl
1	199907	1999-07-01	ATA Airlines	TZ
2	199907	1999-07-01	ATA Airlines	TZ
3	199907	1999-07-01	ATA Airlines	TZ

3. Step 1- Profiling Raw Data

The Databricks DQX profiler was used to analyze column distributions, null ratios, and distinct values.

Approach

The profiling configuration was customized for aviation data:

- Numeric outlier detection enabled

- String trimming enabled
- Sampling used to improve performance

Code Used:

```python
profile_options = {
    "round": True,              # round numeric min/max values
    "max_in_count": 15,         # generate is_in if we have low number of distinct categorical values (airlines/regions)
    "distinct_ratio": 0.05,     # generate is_distinct if we have small distinct ratio
    "max_null_ratio": 0.05,     # allow up to 5% nulls before flagging (some aircraft fields optional)
    "remove_outliers": True,    # remove numeric outliers during profiling
    "outlier_columns": ["Landing_Count", "Total_Landed_Weight"],  # aviation numeric metrics
    "num_sigmas": 3,            # acceptable range within ±3 standard deviations
    "trim_strings": True,       # trim whitespace from airline/aircraft text fields
    "max_empty_ratio": 0.02,    # allow small % of empty strings
    "sample_fraction": 0.3,     # use 30% sample for profiling (faster on larger datasets)
    "sample_seed": None,        # random sampling seed
    "limit": 10000,             # cap rows used for profiling
}
```

```python
# Profile the data
profiler = DQProfiler(ws)

#ways to profile data

summary_stats, profiles = profiler.profile(dfLandStat_trimmed,options=profile_options, columns=columns_to_profile)

# print the data profile and stats of the data

# https://databrickslabs.github.io/dqx/docs/reference/quality_rules/#row-level-quality-checks
#print(profiles)  # Based on the input data a profile is created.
for pf in profiles: # print list
  print(pf)

print("----------------------------------------------------------------------------------------------------")
#print(summary_stats)
```

Output Terminal Debug console pip logs


```
{
    "Activity_Period": {
        "count": 10000,
        "mean": 200932.441,
        "stddev": 598.8692883772331,
        "min": "199907",
        "25%": "200407.0",
        "50%": "200912.0",
        "75%": "201501.0",
        "max": "201908",
        "count_non_null": 10000,
        "count_null": 0
```


> See performance (38)

Outcome

Profiling helped identify:

- GEO Summary domain values
- Numeric ranges for Landing_Count and Total_Landed_Weight
- Potential data inconsistencies

4. Step 2 - Define Data Quality Expectations

Custom DQX rules were created using YAML format. These rules reflect both domain constraints and known tampering scenarios.

Quality Rules Implemented:

Domain Validation

- GEO Summary must be Domestic or International
- GEO Region must belong to known aviation regions

Numeric Validation

- Landing_Count must be between 1 and 5000
- Total_Landed_Weight must be less than 400,000,000

Date Validation

- Activity Period Start Date must not be a future date

Duplicate Detection

Duplicate records were identified using a composite business key:

```python
# Step 1: Define correct duplicate key columns from dataset
key_cols = [
  "Activity_Period",
  "Operating_Airline_IATA_Code",
  "Published_Airline_IATA_Code",
  "GEO_Summary",
  "GEO_Region",
  "Landing_Aircraft_Type",
  "Aircraft_Model"
]
#Rows differ mainly by: Aircraft Model / Version, GEO Region, Airline codes
```

```python
#Step 2: Add Duplicate Detection Column to compute dup_count
from pyspark.sql.window import Window
from pyspark.sql.functions import count

w = Window.partitionBy(*key_cols)

dfLandStat_qc = dfLandStat_trimmed.withColumn(
    "dup_count",
    count("*").over(w)
)
```

```
> dfLandStat_qc: pyspark.sql.connect.dataframe.DataFrame = [Activity_Period: string, Activity_Period_Start_Date: date ...
14 more fields]
```

Columns profiled and the profile_options implemented

```python
import json

profile_options = {
    "round": True,            # round numeric min/max values
    "max_in_count": 15,       # generate is_in if we have low number of distinct categorical values (airlines/regions)
    "distinct_ratio": 0.05,   # generate is_distinct if we have small distinct ratio
    "max_null_ratio": 0.05,   # allow up to 5% nulls before flagging (some aircraft fields optional)
    "remove_outliers": True,  # remove numeric outliers during profiling
    "outlier_columns": ["Landing_Count", "Total_Landed_Weight"],  # aviation numeric metrics
    "num_sigmas": 3,          # acceptable range within ±3 standard deviations
    "trim_strings": True,     # trim whitespace from airline/aircraft text fields
    "max_empty_ratio": 0.02,  # allow small % of empty strings
    "sample_fraction": 0.3,   # use 30% sample for profiling (faster on larger datasets)
    "sample_seed": None,      # random sampling seed
    "limit": 10000,           # cap rows used for profiling
}

# if only specific columns to profile
columns_to_profile = [
  "Activity_Period",
  "Activity_Period_Start_Date",
  "Operating_Airline",
  "Operating_Airline_IATA_Code",
  "Published_Airline",
  "Published_Airline_IATA_Code",
  "GEO_Summary",
  "GEO_Region",
  "Landing_Aircraft_Type",
  "Aircraft_Body_Type",
  "Aircraft_Manufacturer",
  "Aircraft_Model",
  "Aircraft_Version",
```

```python
#db_fs = ws.dbutils.fs.ls('/')
#for f in db_fs:
#  print(f.path)
#print
("-----------------------------------------------------------------------------------------------------")

# Profile the data
profiler = DQProfiler(ws)

#ways to profile data

summary_stats, profiles = profiler.profile(dfLandStat_trimmed,options=profile_options,
columns=columns_to_profile)

# print the data profile and stats of the data

# https://databrickslabs.github.io/dqx/docs/reference/quality_rules/#row-level-quality-checks
#print(profiles)  # Based on the input data a profile is created.
for pf in profiles: # print list
  print(pf)

print
("-----------------------------------------------------------------------------------------------------")
#print(summary_stats)
# for key, value in summary_stats.items(): #print dictionary
#    print (key,value)

# try printing in a formated way
json_formatted = json.dumps(summary_stats, indent=4, default=str)
print(json_formatted)
print
("-----------------------------------------------------------------------------------------------------")
```

5. Step 3 - Detect Bad Records

The DQX engine was used to apply the defined rules and split the dataset into:

- Valid records
- Quarantined records

Code Execution:

05:42 PM (8s) 16 Python

```python
#Step 4: DETECT BAD RECORDS (DQX ENGINE)
from databricks.labs.dqx.engine import DQEngine
from databricks.sdk import WorkspaceClient

ws = WorkspaceClient()
dqengine = DQEngine(ws)

valid, quarantine = dqengine.apply_checks_by_metadata_and_split(
    dfLandStat_qc,
    udChecks,
    globals()
)

display(quarantine)
display(valid)
```

See performance (2)

valid: pyspark.sql.connect.dataframe.DataFrame = [Activity_Period: string, Activity_Period_Start_Date: date ... 14 more fields]

quarantine: pyspark.sql.connect.dataframe.DataFrame = [Activity_Period: string, Activity_Period_Start_Date: date ... 16 more fields]

Table

	Activity_Period	Activity_Period_Start_Date	Operating_Airline	Operating_Airline_I
1	199907	1999-07-01	Air Canada	AC
2	199907	1999-07-01	Air Canada	AC
3	199907	1999-07-01	Air Canada	AC
4	199907	1999-07-01	China Airlines	CI
5	199907	1999-07-01	China Airlines	CI
6	199907	1999-07-01	China Airlines	CI

Results

The rule failure analysis showed:

- Majority failures caused by duplicate records
- Minor failures due to future dates and invalid numeric ranges

OUTPUT: output table from Which rules fail most

05:43 PM (3s) 19 Python

```python
#Step 5: Which rules fail most
from pyspark.sql import functions as F

display(
  quarantine
    .select(F.explode("_errors").alias("e"))
    .groupBy(F.col("e.name").alias("rule"))
    .count()
    .orderBy(F.desc("count"))
)
```

See performance (1)

Table

	rule	count
1	not_dup_count_1	11519
2	not_activity_period_start_date_is_not_null_and_activity_period_start_date_current_date	1
3	not_landing_count_is_not_null_and_landing_count_between_1_and_5000	1

6. Step 4 - Route Failed Data

Failed rows were routed to a quarantine dataset while valid records were saved separately.

Final Counts
Total rows: 44994
Valid rows saved: 38157
Quarantine rows saved: 11520

Output of Final Data Quality Results:

```python
# Print results
print("FINAL DATA QUALITY RESULTS")
print("-----------------------------------")
print("Total input rows:", dfLandStat_trimmed.count())
print("Valid rows saved:", valid2.count())
print("Quarantine rows saved:", quarantine.count())
print("-----------------------------------")

print("Valid dataset preview:")
display(valid2.limit(20))

print("Quarantine dataset preview:")
display(quarantine.limit(20))
```

> See performance (5)

```
FINAL DATA QUALITY RESULTS
-----------------------------------
Total input rows: 44994
Valid rows saved: 38157
Quarantine rows saved: 11520
-----------------------------------
Valid dataset preview:
```

Table

	Activity_Period	Activity_Period_Start_Date	Operating_Airline	Operating_Airli
1	12/31/2099	1999-07-01	ATA Airlines	TZ
2	199907	1999-07-01	Union Flights	null
3	199907	1999-07-01	Emery Worldwide Airlines, Inc.	null
4	199907	1999-07-01	DHL Express (USA)	null
5	199907	1999-07-01	Odyssey Transport	null
6	199907	1999-07-01	Sports Jet	null

Valid dataset preview:

```
Valid dataset preview:
```

Table

	Activity_Period	Activity_Period_Start_Date	Operating_Airline	Operating_Airline_I
1	12/31/2099	1999-07-01	ATA Airlines	TZ
2	199907	1999-07-01	Union Flights	null
3	199907	1999-07-01	Emery Worldwide Airlines, Inc.	null
4	199907	1999-07-01	DHL Express (USA)	null
5	199907	1999-07-01	Odyssey Transport	null
6	199907	1999-07-01	Sports Jet	null
7	199907	1999-07-01	Atlas Air, Inc. (inactive)	null
8	199907	1999-07-01	China Eastern Airlines, Inc	null
9	199907	1999-07-01	Aeroflot Russian International Airlin...	null
10	199907	1999-07-01	Aeroflot Russian International Airlin...	null
11	199907	1999-07-01	ATA Airlines	-999999
12	199907	1999-07-01	American Airlines	AA
13	199907	1999-07-01	American Airlines	AA

See performance (5)

Output Terminal Debug console pip logs

Table

	raft_Model	Aircraft_Version	Landing_Count	Total_Landed_Weight	dup_count
1		0	71	25418000	1
2		B	21	163800	1
3		100	48	6840000	1
4		73	23	6325000	1
5		0	44	616000	1
6		400	1	124000	1
7		200	5	3150000	1
8		0	8	3519968	1
9		0	9	2879955	1
10		UNKNOWN	4	1543220	1
11		UNKNOWN	78	15444000	1
12		W	184	36432000	1
13		200ER	115	31970000	1
14		0	32	11129600	1
15		SP	377	49010000	1

20 rows | 4.96s runtime

```
Quarantine dataset preview:
```

See performance (5)

Quarantine dataset preview:

Output Terminal Debug console pip logs

```
Quarantine dataset preview:
```

Table

	Activity_Period	Activity_Period_Start_Date	Operating_Airline	Operating_Airline_IATA_Cod
2	199907	1999-07-01	Air Canada	AC
3	199907	1999-07-01	Air Canada	AC
4	199907	1999-07-01	China Airlines	CI
5	199907	1999-07-01	China Airlines	CI
6	199907	1999-07-01	China Airlines	CI
7	199907	1999-07-01	China Airlines	CI
8	199907	1999-07-01	China Airlines	CI
9	199907	1999-07-01	China Airlines	CI
10	199907	1999-07-01	China Airlines	CI
11	199907	1999-07-01	Delta Air Lines	DL
12	199907	1999-07-01	Delta Air Lines	DL
13	199907	1999-07-01	Delta Air Lines	DL
14	199907	1999-07-01	Lufthansa German Airlines	LH
15	199907	1999-07-01	Lufthansa German Airlines	LH
16	199907	1999-07-01	ATA Airlines	TZ

```
Quarantine dataset preview:
```

Table

	_errors
1	array 0: {"name": "not_dup_count_1", "message": "Duplicate landing record detected", "columns": null, "filter": null, "function": "sql_expression", "run_time": "2026-02-16T22:46:31.871Z", "run_id": "2ef5592d-4ca5-45f3-b218-fa1ef0965b3c", "user_metadata": {}}
2	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
3	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
4	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
5	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
6	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
7	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
8	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
9	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
10	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
11	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
12	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...
13	[{"name":"not_dup_count_1","message":"Duplicate landing record detected","columns":null,"filter":null,"function":"sql_expr...

20 rows | 4.96s runtime

Output Terminal Debug console pip logs

```
Quarantine dataset preview:
```

Table

	Aircraft_Manufacturer	Aircraft_Model	Aircraft_Version	Landing_Count	Total_Lai
1	Boeing	767	209	4	
2	Boeing	767	233	26	
3	Boeing	767	333	1	
4	Boeing	747	0	2	
5	Boeing	747	200	4	
6	Boeing	747	230	1	
7	Boeing	747	0	2	
8	Boeing	747	200	2	
9	Boeing	747	243	2	
10	Boeing	747	400	31	
11	Lockheed	L1011	100	63	
12	Lockheed	L1011	250	4	

20 rows | 4.96s runtime

See performance (5)

7. Step 5 - Data Remediation

A remediation pipeline was implemented to improve data quality by:

- Removing duplicate records using business key
- Filtering future dates
- Enforcing numeric ranges
- Filling optional aircraft fields with default values

This resulted in a significantly cleaner dataset ready for downstream analytics.

```python
#Step 6: REMEDIATE DATA
#We now clean the dataset: Remove duplicates, Remove future dates, Keep valid numeric ranges, Fill optional text fields

from pyspark.sql.functions import col, current_date

dfRemediated = (
  dfLandStat_trimmed
    .filter(col("Activity_Period_Start_Date").isNotNull())
    .filter(col("Activity_Period_Start_Date") <= current_date())
    .filter(col("Landing_Count").between(1,5000))
    .filter(col("Total_Landed_Weight") <= 400000000)
    .dropDuplicates(key_cols)
    .fillna({
        "Aircraft_Version": "UNKNOWN",
        "Aircraft_Model": "UNKNOWN",
        "Aircraft_Manufacturer": "UNKNOWN"
    })
)
```

Output:

```
Output   Terminal   Debug console   pip logs

dfRemediated: pyspark.sql.connect.dataframe.DataFrame
    Activity_Period: string
    Activity_Period_Start_Date: date
    Operating_Airline: string
    Operating_Airline_IATA_Code: string
    Published_Airline: string
    Published_Airline_IATA_Code: string
    GEO_Summary: string
    GEO_Region: string
    Landing_Aircraft_Type: string
    Aircraft_Body_Type: string
    Aircraft_Manufacturer: string
    Aircraft_Model: string
    Aircraft_Version: string
    Landing_Count: integer
    Total_Landed_Weight: double
```

Code block after data remediation:

```python
# Step 7 — RECHECK AFTER REMEDIATION
dfRemediated_qc = dfRemediated.withColumn(
    "dup_count",
    count("*").over(Window.partitionBy(*key_cols))
)

valid2, quarantine2 = dqengine.apply_checks_by_metadata_and_split(
    dfRemediated_qc,
    udChecks,
    globals()
)

print("After remediation:")
print("Valid rows:", valid2.count())
print("Remaining quarantine:", quarantine2.count())
```

Output of above code block:

```
    Activity_Period_Start_Date: date
    Operating_Airline: string
    Operating_Airline_IATA_Code: string
    Published_Airline: string
    Published_Airline_IATA_Code: string
    GEO_Summary: string
    GEO_Region: string
    Landing_Aircraft_Type: string
    Aircraft_Body_Type: string
    Aircraft_Manufacturer: string
    Aircraft_Model: string
    Aircraft_Version: string
    Landing_Count: integer
    Total_Landed_Weight: double
    dup_count: long
quarantine2: pyspark.sql.connect.dataframe.DataFrame
    Activity_Period: string
    Activity_Period_Start_Date: date
    Operating_Airline: string
    Operating_Airline_IATA_Code: string
    Published_Airline: string
    Published_Airline_IATA_Code: string
    GEO_Summary: string
    GEO_Region: string
    Landing_Aircraft_Type: string
    Aircraft_Body_Type: string
    Aircraft_Manufacturer: string
    Aircraft_Model: string
    Aircraft_Version: string
    Landing_Count: integer
    Total_Landed_Weight: double
    dup_count: long
    _errors: array
        element: struct
            name: string
            message: string
            columns: array
                element: string
```

```
valid2: pyspark.sql.connect.dataframe.DataFrame
    Activity_Period: string
    Activity_Period_Start_Date: date
    Operating_Airline: string
    Operating_Airline_IATA_Code: string
    Published_Airline: string
    Published_Airline_IATA_Code: string
    GEO_Summary: string
    GEO_Region: string
    Landing_Aircraft_Type: string
    Aircraft_Body_Type: string
    Aircraft_Manufacturer: string
    Aircraft_Model: string
    Aircraft_Version: string
    Landing_Count: integer
    Total_Landed_Weight: double
    dup_count: long

After remediation:
Valid rows: 38157
Remaining quarantine: 0
```

8. Observations

Key findings from profiling and validation include:

8.1 Duplicate Records Were the Primary Quality Issue

The rule failure analysis showed that the majority of quarantined records were caused by duplicate entries based on the defined business key:

Activity_Period

Operating_Airline_IATA_Code

Published_Airline_IATA_Code

GEO_Summary

GEO_Region

Landing_Aircraft_Type

Aircraft_Model

Out of 44,994 total rows:

- **11,519 rows failed the duplicate rule**
- This represents the largest source of data quality violations

This indicates that the dataset was intentionally tampered to include repeated landing records rather than widespread domain or format errors.

8.2 Domain Values Were Largely Consistent

Profiling confirmed that:

- **GEO_Summary** only contained Domestic and International
- **GEO_Region** values aligned with known aviation regions such as US, Europe, Asia, and Middle East

Very few records violated domain expectations, indicating that categorical fields were mostly well-structured.

8.3 Numeric Ranges Reflected Realistic Aviation Patterns

From profiling:

- Landing_Count ranged approximately between **1 and ~3000**
- Total_Landed_Weight ranged up to approximately **312 million**

Only **one record** violated the numeric range rule, suggesting that most tampering focused on duplication rather than numeric distortion.

8.4 Future Dates Were Rare but Detected

The date validation rule identified a small number of future-dated records. These were successfully isolated into the quarantine dataset, demonstrating that DQX rules correctly enforced temporal constraints.

8.5 Data Type Issues Were Successfully Remediated During Trimming

Raw dataset values included:

- Numeric fields stored as strings with commas (e.g., “1,533”)

- Date fields stored as text

During the trimming stage:

- Numeric columns were cleaned and cast to integer/double
- Date fields were parsed using the correct format

As a result, invalid data types appeared as nulls and were captured by DQX validation rules.

8.6 Overall Data Quality Assessment

After applying DQX validation:

- **38,157 rows were validated as clean**
- **11,520 rows were routed to quarantine**

The dataset quality issues were concentrated in specific rule categories rather than evenly distributed across all columns.

This demonstrates that rule-based validation is effective at isolating targeted anomalies without over-filtering valid operational data.

9. Data Quality Architecture:
Raw Dataset (Bronze)
↓
Trimmed Dataset (Silver)
↓
DQX Validation Engine
↓
Valid Dataset **Quarantine Dataset**

10. Conclusion

This workflow implemented a complete DQX-based data quality pipeline for aviation landing data. The process included profiling, rule definition, validation, remediation, and routing of failed records.

The workflow ensures that only high-quality data moves forward into the refined layer while maintaining transparency through quarantine datasets.